

Technical Paper

A branch and bound method for the line balancing problem in U-shaped assembly lines with equipment requirements

Dilek Ogan^a, Meral Azizoglu^{b,*}

^a ASELSAN Electronics Company, Ankara, Turkey

^b Department of Industrial Engineering, Middle East Technical University, Ankara 06800, Turkey

ARTICLE INFO

Article history:

Received 14 December 2013

Received in revised form

20 December 2014

Accepted 21 February 2015

Available online 3 April 2015

Keywords:

U-shaped assembly line

Equipment requirement

Branch and bound algorithm

ABSTRACT

In this study we consider a U-shaped assembly line balancing problem where each task uses a specified set of equipments and each type of equipment has a specified cost. Our problem is to assign the tasks together with their equipments to the workstations so as to minimize the total equipment cost. We formulate the problem as a mixed integer linear programming model that is capable of solving small sized instances. We propose a branch and bound algorithm that uses efficient precedence relations and lower bounds. We find that the algorithm is able to solve moderate sized problem instances in reasonable times.

© 2015 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights reserved.

1. Introduction

Assembly lines are flow oriented production systems where different parts of a product are assembled through a series of workstations. Those lines have great importance in the industrial production of highly standardized commodities. Due to the high investment and running costs involved, the design of such lines is of considerable importance [1].

In assembly line terminology, a task is defined as the smallest, indivisible work element. The tasks are assigned to the workstations while taking the precedence relations and product demand into account. The precedence relations define technological restrictions, i.e., some tasks cannot start before some others complete. To guarantee the satisfaction of the demand, a product is to be completed at the end of each specified period, so called the cycle time and the sum of the operation times of the tasks cannot exceed the cycle time.

Two basic assembly line layouts, namely straight line layouts and U-shaped line layouts, are considered in the literature. In straight lines, operators perform tasks on a continuous portion of the line and a task can be performed only if all its predecessors are complete. On the other hand, in U-shaped lines (see Fig. 1) a task can be performed either all its predecessors or all its successors are complete.

Operators can handle only adjacent tasks in straight lines whereas in U-shaped lines they are also allowed to work across both legs as shown in Fig. 1.

In today's competitive environment, the need for higher productivity with greater flexibility has advocated the need for the U-shaped lines. Miltenburg's [2] study on more than 100 Japanese and US companies have revealed that the U-shaped lines have significant practical importance in reducing the work-in-process levels and improving the productivity measures. A high level of participation of the workers in U-shaped lines improves productivity. The U-shaped line operators are expected to have higher skill levels as they perform a wider range of the tasks. This provides quick response to the changing environments due to the possibility of re-allocating the skilled operators, hence brings higher flexibility. The line can be rebalanced easily by adding/removing workers hence the changes in the required production rates can be adapted easily. These issues are discussed thoroughly in Monden [3] in the context of Toyota Production System.

The majority of the literature on the assembly lines assumes that the tasks require only one worker at each station. But in practice some tasks cannot be performed by one worker and some tasks may require additional assistance or a particular resource. Each additional worker, assistance and resource may be treated as equipment requirements. Ağpak and Gökçen [4] discuss a straight line manufacturing environment with limited cutting tools that are required by some tasks. As they state, the equipments, in particular the automated ones (like robots), may be of high value so that their proper allocation to the workstations is of great importance for an efficient

* Corresponding author. Tel.: +90 3122102281.

E-mail address: ma@metu.edu.tr (M. Azizoglu).

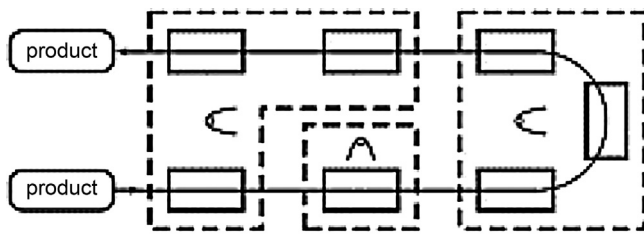


Fig. 1. A U-shaped line.

Taken from [1].

operation of the assembly line. Kara et al. [5] mention a real application at a solar collector producer where some tasks require more than one equipment (e.g. worker and assistant) and some tasks have equipment alternatives (e.g. assistant versus vacuum gripper) where each alternative has a cost and costly alternative reduces the task time. Faaland et al. [6] refer to the problems with equipment alternatives as resource dependent assembly line balancing problem.

In this study, we assume that each task of a U-shaped assembly line, requires a defined set of equipments (no equipment alternative exists), like an additional worker, an assistant and a particular resource. Each task has a fixed (equipment independent) time and each equipment has a fixed cost. The cost of an equipment can be attributed to its market value, labor cost and usage cost. The line is already mounted with specified number of workstations. Our problem is to assign the tasks together with their equipments to the workstations so as to minimize the total equipment cost. A good solution to our model would assign the tasks that are performed by the same equipment to the same workstation.

To the best of our knowledge there is no optimization study that considers equipment assignments in U-shaped lines, despite its practical importance. The problem is theoretically more challenging than its straight line counterpart, due to having more choices for task and equipment assignments to any workstation. We use this challenge and propose an optimization algorithm that can solve the problem instances up to 21 tasks and 10 workstations, and 30 tasks and 8 workstations, in 2 h. Miltenburg and Wijngaard [7] state that most U-shaped assembly lines consist of less than 30 tasks and 10 stations. We hope that our algorithm will be considered by the practitioners that operate U-shaped lines with several equipment types.

The rest of the paper is organized as follows. Section 2 presents a brief review of the literature. In Section 3 we define our problem, give its mathematical formulation and settle the complexity. We introduce our branch and bound algorithm with the reduction and bounding mechanisms in Section 4. In Section 5, we present our computational experiment and discuss its results. We conclude in Section 6, by stating the main results and offering suggestions for future research.

2. Literature review

In the section, we give brief reviews of the U-shaped lines and assembly lines with resource (equipment) requirements. Battaia and Dolgui [1] develop a taxonomy to analyze the assembly lines and discuss the recent trends on various line balancing problems. For the detailed reviews on U-shaped lines together with some related manufacturing experiences, we refer the reader to Miltenburg [2].

The majority of the U-shaped line balancing problems considers the problem of minimizing the number of the workstations subject to a fixed cycle time. Some noteworthy of those studies are due to Miltenburg and Wijngaard [7], Urban [8], Aase et al. [9], Erel et al. [10] and Gökçen et al. [11]. Miltenburg and Wijngaard [7] propose

a dynamic programming algorithm that could handle small-sized instances. Urban [8] proposes an integer programming formulation that uses a phantom network that allows forward and backward assignment of tasks to workstations. Aase et al. [9] develop a branch and bound procedure that uses several branching strategies and dominance rules. They show that their procedure is significantly faster than the integer program of Urban [8]. Erel et al. [10] develop a new simulated annealing based algorithm and Gökçen et al. [11] provide a shortest route formulation. Lian et al. [12] and Hamzadayi and Yildiz [13] consider some extensions of the U-shaped line balancing problem. Lian et al. [12] and Hamzadayi and Yildiz [13] study mixed model U-shaped assembly line balancing and sequencing problems.

Recently due to the fast-changing customer demands, assembly tasks having equipment requirements, have gained considerable importance. Attributing to its practical importance, the research on the assembly lines with equipment considerations has been grown in the recent years. Buckhin and Tzur [14] study a line balancing problem where each task requires a single equipment and there are several equipment alternatives. They assume a single equipment can be assigned to each workstation and their objective is to minimize the total equipment cost. Buckhin and Rubinovitz [15] study the assembly line balancing problem with station paralleling and equipment selection decisions. They show the parallel station problem can be solved by the branch and bound algorithm developed by Buckhin and Tzur [14]. Barutçuoğlu and Azizoglu [16], study the equipment selection and line balancing problem with fixed number of workstations. They aim to minimize the total equipment cost and develop a branch and bound algorithm. Ege et al. [17] consider the assembly line balancing problem with parallel workstations. They assume each task requires a specified equipment and aim to minimize the sum of station opening and equipment costs. They propose an exact and heuristic branch and bound algorithms. Yoosefelahi et al. [18] consider a multi-criteria assembly line balancing problem with equipment decisions and propose evolution strategy algorithms.

Kara et al. [5] and Jayaswal and Agarwal [19] consider the equipment dependent task times such that there are equipment alternatives for each task. Their problem is simultaneously selecting the equipments and assigning the tasks with their selected equipments to the workstations. Jayaswal and Agarwal [19] propose a simulated annealing approach with promising results however with no guarantee of optimality. Kara et al. [5] assume that the available number of equipments is limited and in their experiments they set this limit to either 1 or 2 for each equipment and their study is limited to a mathematical model. In our U-shaped problem with no equipment alternatives, we observe that the mathematical models' performances are more sensitive to the number of the equipments and they cannot be of much help to solve larger-sized real life problem instances.

3. Problem definition, mathematical model and the complexity

Consider a single product that has to be assembled on a U-shaped line having K workstations. There is a target production rate that specifies the cycle time, CT . The product requires N indivisible tasks each of which should be assigned to one of the workstations. There are L equipment types, a pre-specified subset of which is required to process each task. All tasks can be performed in all workstations and all equipments can be assigned to all workstations.

Our problem is to assign the tasks together with their required equipments to the workstations so as to minimize the total

equipment cost. En route to minimizing the total cost the tasks that can be performed by the same equipment are likely to be assigned to the same workstation.

Our parameters are as defined below:

- t_i : processing time of task i $i = 1, 2, \dots, N$
 EC_l : cost of equipment l $l = 1, 2, \dots, L$
 l_i : set of equipments required by task i $i = 1, 2, \dots, N$
 P_i : predecessor set of task i $i = 1, 2, \dots, N$
 SC_i : successor set of task i $i = 1, 2, \dots, N$

The variables that explain our decisions are as stated below:

$$X_{ik} = \begin{cases} 1, & \text{if task } i \text{ is assigned to workstation } k \\ 0, & \text{otherwise} \end{cases} \quad i = 1, 2, \dots, N \quad k = 1, 2, \dots, K$$

$$Z_{lk} = \begin{cases} 1, & \text{if equipment } l \text{ is assigned to workstation } k \\ 0, & \text{otherwise} \end{cases} \quad l = 1, 2, \dots, L \quad k = 1, 2, \dots, K$$

$$Y_i = \begin{cases} 1, & \text{if all predecessors of task } i \text{ are assigned to the same or earlier stations} \\ 0, & \text{if all successors of task } i \text{ are assigned to the same or earlier stations} \end{cases} \quad i = 1, 2, \dots, N$$

We now give the mathematical representation of our problem.

$$\text{Min} \sum_{l=1}^L \sum_{k=1}^K EC_l Z_{lk} \quad (1)$$

$$X_{ik} \leq \frac{\sum_{j \in SC(i)} \sum_{r \leq k} X_{jr}}{|SC_i|} + Y_i \quad \forall i, k \quad (2)$$

$$X_{ik} \leq \frac{\sum_{j \in P(i)} \sum_{r \leq k} X_{jr}}{|P_i|} + (1 - Y_i) \quad \forall i, k \quad (3)$$

$$\sum_{k=1}^K X_{ik} = 1 \quad \forall i \quad (4)$$

$$\sum_{i=1}^N t_i X_{ik} \leq CT \quad \forall k \quad (5)$$

$$X_{ik} \leq \frac{\sum_{l \in l_i} Z_{lk}}{|l_i|} \quad \forall i, k \quad (6)$$

$$X_{ik} \in \{0, 1\} \quad \forall i, k \quad (7)$$

$$Z_{lk} \in \{0, 1\} \quad \forall l, k \quad (8)$$

$$Y_i \in \{0, 1\} \quad \forall i \quad (9)$$

The objective function (1) minimizes the total equipment cost over all workstations.

Constraint Sets (2) and (3) together state that a task is assigned to a workstation only if either all its successors or all its predecessors are assigned to that workstation or to one of the earlier workstations. Constraint Set (3) ensures that whenever task i is assigned to a workstation k , $X_{ik} = 1$ and all its predecessors are assigned to workstations 1 through k , i.e., $(\sum_{j \in SC(i)} \sum_{r \leq k} X_{jr}) / |SC_i| = 1$ then Y_i takes value 1. Otherwise Y_i takes value 0 and Constraint Set (3) ensures that then all successors of task i such that $X_{ik} = 1$, are assigned to workstations 1 through k , i.e., $(\sum_{j \in P(i)} \sum_{r \leq k} X_{jr}) / |P_i| = 1$. Constraint Set (4) ensures that each task is assigned to exactly one workstation. Constraint Set (5) states that the sum of the processing times of the tasks assigned to any workstation does not exceed the cycle time. Constraint Set (6) ensures that if a task is assigned to a workstation, its equipment set should also be assigned to that workstation. Constraint Sets (7) and (8) are for the binary constraints on the task assignment and equipment assignment

variables, respectively. Constraint Set (9) are for the binary indicator variables used in Constraint Sets (2) and (3).

Theorem 1 defines the complexity of the problem.

Theorem 1. *The U shaped line balancing problem with equipment requirements is strongly NP-hard.*

Proof. Assume a special case of the problem with a single equipment, say equipment r . That is $l_i = \{r\}$ for all i . For this special case, the objective function reduces to $EC_r \sum_{k=1}^K Z_{rk}$ which is equivalent to $\sum_{k=1}^K Z_{rk}$. Such an assignment may keep some workstations idle

en route to reducing total equipment cost. As all workstations use the same equipment r , $\sum_{k=1}^K Z_{rk}$ gives the number of workstations with task assignments. The problem of minimizing the number of workstations with task assignments given the cycle time is strongly NP-hard (see [20]), so is our problem with additional complexity brought by the U-shape and equipment assignment decisions. $\square$

4. The branch and bound algorithm

In this study, we propose a branch and bound algorithm for the optimal assignment of the tasks together with their equipments to each workstation. We aim to find exact solutions attributing to the practical and theoretical importance of our problem.

Given a partial sequence with the first k workstations opened, let S be the set of not yet assigned tasks. We say a task in S is eligible to the current workstation if either of the following conditions holds.

- all predecessor tasks are already assigned
- all successor tasks are already assigned

We index the tasks such that $i < j$ implies task i is not successor of task j .

To avoid the duplication of the solutions, for the selected task, we open branches to the nodes representing the tasks having higher indices and that become eligible after the most recent task assignment and a node representing closing the current workstation.

At level k , for each task i , the nodes of level $k - 1$ such that $i < j$ and the nodes that become eligible after assigning task i are opened.

Consider the network depicted in Fig. 2. The t_i and l_i values are given at the top of each node.

At level 1, task A and task F are eligible, and level 2 nodes are as shown in Fig. 3.

From task A, four nodes are opened. Node F represents a task with higher index. Nodes B and C represent the tasks that become eligible after assigning task A. Close node represents closing the first workstation and opening the second one. From task F, three nodes are opened. Task D and task E become eligible by assigning task F and there is a close node.

We use the following property to reduce the size of the subproblem. The following property defines the tasks that should be put to one workstation alone.

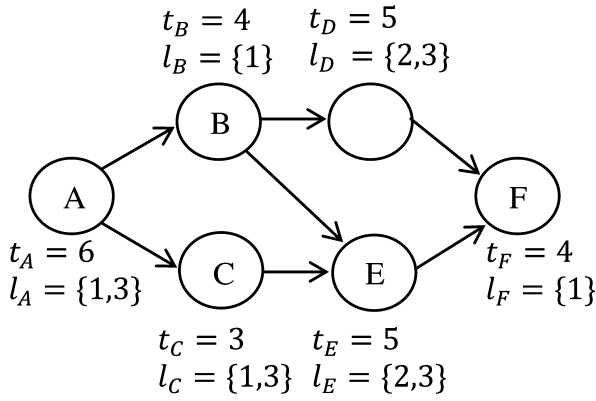


Fig. 2. The precedence diagram of the example instance.

Property 1. If $t_j + \min_{i \in S} \{t_i\} > CT$ then put task j to one workstation alone.

The proof follows from the fact that assigning task j even with the shortest task violates the cycle time constraint.

We check [Property 1](#) at each close node. If there exists a task j that satisfies the conditions of the property then we update $K = K - 1$. Such a task is assigned when all its predecessors or successors are assigned.

At each close node, we check for the feasibility posed by the number of the workstations. Given already closed k workstations with set S of unassigned tasks, $\lceil \sum_{i \in S} t_i / CT \rceil$ is a lower bound on the number of remaining workstations.

[Property 2](#) states the feasibility check formally.

Property 2. If $k + \lceil \sum_{i \in S} t_i / CT \rceil > K$ then the close option cannot lead to a feasible solution.

Proof. There are K workstations, k of which are already used. Once workstation k is closed, the remaining tasks can be performed using at least $\lceil \sum_{i \in S} t_i / CT \rceil$ workstations. Hence if $\lceil k + \sum_{i \in S} t_i / CT \rceil > K$, the current partial solution cannot produce a feasible solution. $\square$

En route to finding all optimal solutions, we disregard some feasible solutions that cannot lead to an optimal solution. The following property helps us to define such nonpromising solutions. We call a task fittable, if it fits to the current workstation without violating cycle time and precedence constraints.

Property 3. If there is a fittable task with no extra equipment requirement or requiring extra equipments that are not required by any unscheduled task then closing workstation k cannot lead to a unique optimal solution.

Proof. Assume workstation k is closed although there is a fittable task j that satisfies the conditions of the property and task j is assigned to workstation $k+r$. Task j is removed from its current workstation and put into workstation k such a change never increases the number of workstations and total equipment cost

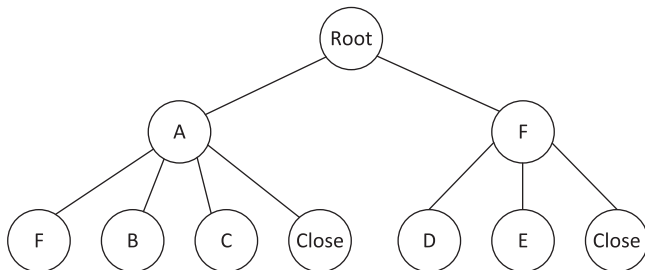


Fig. 3. The branching tree of the example instance.

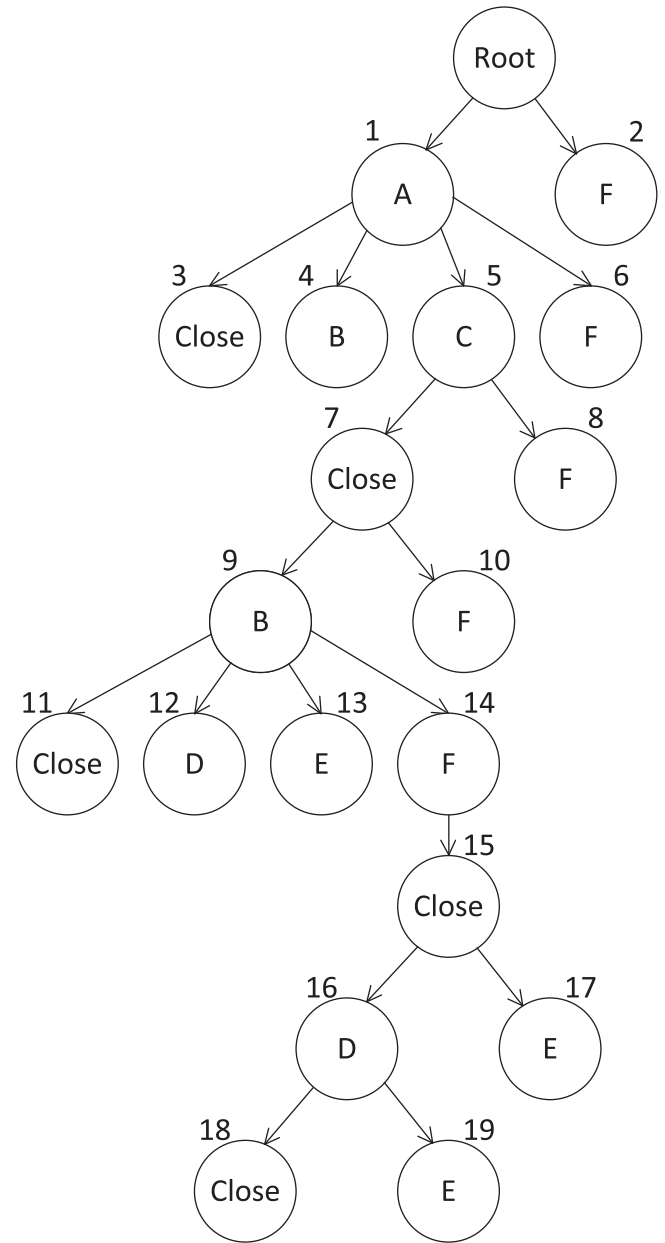


Fig. 4. A partial branch and bound tree for the example instance.

since task j fits to workstation k with no extra equipment i.e., cost, or the extra equipments are not required by any task in workstation $k+r$. Hence a solution in which task j is assigned to workstation k is never worse. $\square$

We propose lower bounds used to eliminate the nonpromising partial solutions. A lower bound on the number of workstations that use equipment r can be stated as $\lceil \sum_{r \in l_i} t_i / CT \rceil$. This follows a valid lower bound on the total equipment cost is $\sum_r EC_r \lceil \sum_{r \in l_i} t_i / CT \rceil$. We state this result formally through [Theorem 2](#).

Theorem 2. $\sum_r EC_r \lceil \sum_{r \in l_i} t_i / CT \rceil$ is a valid lower bound on the optimal total equipment cost.

Proof. Assume Z^* is the optimal equipment cost and Z_r^* is the optimal number of equipment type r , i.e., $Z^* = \sum_r EC_r Z_r^*$.

Table 1
The BAB algorithm and model performances for $L=2$, $N=21$.

CT	K	BAB					Model		
		CPU time		Number of nodes		# of unsolved instances	CPU time		# of unsolved instances
		Aver	Max	Aver	Max		Aver	Max	
Set I									
14	8	19.6	51.6	148,321	395,686	0	495.7	1298.7	0
	9	113.2	443.8	957,564	4,110,762	0	1415.3	2836.8	0
	10	1254.2	5327.6	11,960,447	50,472,787	0	4150.3	7200.0	4
15	8	15.0	67.2	147,048	702,898	0	313.9	565.7	0
	9	18.7	68.3	192,040	733,559	0	2843.2	7200.0	1
	10	23.8	83.8	250,652	873,140	0	4335.6	7200.0	5
21	6	7.3	30.0	72,728	309,043	0	94.1	344.2	0
	7	14.1	43.9	147,302	463,230	0	317.0	590.7	0
	8	14.5	44.6	152,103	474,784	0	1050.5	1732.7	0
Set II									
14	6	19.5	51.7	148,426	395,686	0	362.5	1020.1	0
	7	113.4	434.0	969,726	4,110,762	0	1936.9	7200.0	1
	8	1852.3	7200.0	17,698,624	67,116,686	1	4289.1	7200.0	4
15	6	15.1	67.1	147,048	702,898	0	456.3	781.9	0
	7	19.1	69.6	192,309	733,559	0	2034.2	7200.0	1
	8	24.2	84.8	250,944	873,140	0	4679.7	7200.0	4
21	6	7.6	30.3	74,126	309,054	0	131.7	641.9	0
	7	14.4	44.1	148,926	463,278	0	398.0	903.5	0
	8	14.7	44.9	153,728	474,835	0	1776.2	5062.9	0

$Z_r^* \geq \left\lceil \sum_{r \in l_i} t_i / CT \right\rceil$ follows that $\sum_r EC_r Z_r^* \geq \sum_r EC_r \left\lceil \sum_{r \in l_i} t_i / CT \right\rceil$.
Hence $Z^* \geq \sum_r EC_r \left\lceil \sum_{r \in l_i} t_i / CT \right\rceil$. $\square$

We extend the lower bound to each partial schedule S , using the following notation.

S_k = set of tasks assigned to the current workstation k
 W_k = load of the current workstation k

The following two cases arise:

Case 1. Equipment r is not already assigned to the current workstation then

$$LB_r = \left\lceil \sum_{\substack{i \in S \\ r \in l_i}} t_i / CT \right\rceil \text{ is a lower bound on the number of the}$$

workstations that reside equipment r .

Case 2. Equipment r is already assigned to the current workstation then

$$LB_r = \left\lceil \sum_{\substack{i \in S \\ r \in l_i \\ i \notin S_k}} t_i - (CT - W_k) / CT \right\rceil \text{ is a lower bound on the}$$

number of the workstations that reside equipment r .

We assume that the slack time of the current workstation is used by the tasks that use equipment r . $\sum_r EC_r LB_r$ is a lower bound on

the total cost of unassigned tasks. We eliminate the partial schedule whenever $TC(S) + \sum_r EC_r LB_r \geq UB$ where

$TC(S)$ = total equipment cost incurred in workstations 1 through k
 UB = total equipment cost of the best known feasible solution

Our initial experimentation has shown that the initial upper bound UB does not have any significant effect on the performance. Hence, initially we set UB to a very big value, and update UB whenever a complete solution with a smaller total equipment cost is reached.

We employ a depth-first strategy due to its relatively low memory requirements. According to this strategy we start from root node and explore a branching from the node having the smallest lower bound, i.e., $TC(S) + \sum_r EC_r LB_r$ value. We continue to branching until we reach to the last level with a feasible complete solution or to any level having no feasible or promising partial solutions. If further branching is not possible, we backtrack to the previous level. We terminate whenever we reach to level 0. The incumbent solution at termination is the optimal solution.

We illustrate our branch and bound procedure by the 3 equipments instance whose precedence diagram is given in Fig. 2. We let $CT=10$, $K=3$.

The partial branch and bound tree corresponding to our example is depicted in Fig. 4. The execution number is given on the tree at the top of each node. For example, we first search tasks A and F and give numbers 1 and 2.

$LB(i)$ is the lower bound found after appending the eligible task i to the current partial assignment. We calculate $LB(A)$ and $LB(F)$, i.e., the lower bounds for two eligible tasks of level 1 as follows.

$$LB(A) = \left\lceil \frac{t_A + t_B + t_C + t_F}{10} \right\rceil 100 + \left\lceil \frac{t_D + t_E}{10} \right\rceil 200 + \left\lceil \frac{t_A + t_C + t_D + t_E}{10} \right\rceil 300 = 1000$$

$$LB(F) = \left\lceil \frac{t_A + t_B + t_C + t_F}{10} \right\rceil 100 + \left\lceil \frac{t_D + t_E}{10} \right\rceil 200 + \left\lceil \frac{t_A + t_C + t_D + t_E}{10} \right\rceil 300 = 1000$$

$LB(A) = LB(F)$, we assign the task having lower index, i.e., task A.

After the assignment, $S_1 = \{A\}$, $W_1 = 6$, $S = \{B, C, D, E, F\}$ and tasks B and C become eligible in addition to task F. The total cost of the partial assignment is 400. Close node is fathomed as all eligible tasks can be assigned to the current workstation with no additional equipment (see Property 2). The lower bounds of the eligible tasks are found as follows.

$$LB(B) = \left\lceil \frac{t_C + t_F - (10 - 10)}{10} \right\rceil 100 + \left\lceil \frac{(t_D + t_E)}{10} \right\rceil 200 + \left\lceil \frac{t_C + t_D + t_E - (10 - 10)}{10} \right\rceil 300 = 900$$

$$LB(C) = \left\lceil \frac{t_B + t_F - (10 - 9)}{10} \right\rceil 100 + \left\lceil \frac{(t_D + t_E)}{10} \right\rceil 200 + \left\lceil \frac{t_D + t_E - (10 - 10)}{10} \right\rceil 300 = 600$$

$$LB(F) = \left\lceil \frac{t_B + t_C - (10 - 10)}{10} \right\rceil 100 + \left\lceil \frac{(t_D + t_E)}{10} \right\rceil 200 + \left\lceil \frac{t_C + t_D + t_E - (10 - 10)}{10} \right\rceil 300 = 900$$

We assign task C as it has the smallest lower bound. After the assignment we are left with one unit of slack time. We close the current workstation as no additional task can fit in and we make the following feasibility check

$$k + \left\lceil \frac{(t_B + t_D + t_E + t_F)}{CT} \right\rceil = 1 + \left\lceil \frac{(4 + 5 + 5 + 4)}{10} \right\rceil = 3 \leq K = 3$$

The feasibility test is passed, so we continue with the second workstation. Two eligible tasks, B and F have the same lower bound value of 600. Task B which has the lower index is assigned and tasks D and E become eligible. Task F was already eligible and now it is considered as it has higher index than the last assigned task B. Close option is ignored as there are eligible tasks with no additional equipment requirement. The lower bounds of the eligible tasks are calculated as:

$$LB(D) = \left\lceil \frac{t_F - (10 - 9)}{10} \right\rceil \times 100 + \left\lceil \frac{t_E - (10 - 9)}{10} \right\rceil \times 200 + \left\lceil \frac{t_E - (10 - 9)}{10} \right\rceil \times 300 = 600$$

$$LB(E) = \left\lceil \frac{t_F - (10 - 9)}{10} \right\rceil \times 100 + \left\lceil \frac{t_D - (10 - 9)}{10} \right\rceil \times 200 + \left\lceil \frac{t_D - (10 - 9)}{10} \right\rceil \times 300 = 600$$

$$LB(F) = \left\lceil \frac{(t_D + t_E)}{10} \right\rceil \times 200 + \left\lceil \frac{t_D + t_E}{10} \right\rceil \times 300 = 500$$

Task F is assigned to the current workstation as it has the smallest lower bound. After the assignment, the workstation is closed as there is no unassigned task left having higher index than task F. Feasibility check holds, so we continue with the third workstation.

Note that tasks D and E have the same lower bound and the lower indexed task, i.e., task D is selected for assignment.

Thereafter, task E is assigned as it does not require any additional equipment and a feasible solution is obtained with a total cost of 1000 (tasks A and C at workstation 1, tasks B and F at workstation 2 and tasks D and E at workstation 3). We set the upper bound to 1000 and backtrack to node 15 where there is one more alternative, i.e., task E. No eligible task is left once task E is assigned to the third workstation, so the workstation is closed.

We eliminate the branch as the feasibility test fails, i.e.,

$$k + \frac{t_D}{CT} = 3 + \frac{5}{10} = 4 \not\leq K = 3.$$

We then backtrack to node 9 where there are two more alternatives, task D and task E. The lower bound of each task is 600 and each task leads to a total cost of 1000 if assigned to the current workstation.

We fathom alternatives after the following checks.

$$LB(D) = TC(S) + \sum_r EC_r LB_r = 1000 + 600 = 1600 \geq UB = 1000$$

$$LB(E) = TC(S) + \sum_r EC_r LB_r = 1000 + 600 = 1600 \geq UB = 1000$$

We continue in a similar manner and stop when the root node is reached. The optimal equipment and task assignments are depicted in Fig. 5.

5. Computational experiments

We design an experiment to test the performance of the branch and bound algorithm and the complexity of the mathematical model. We code the branch and bound algorithm in C++ programming language. The model is built using GAMS 23.5.1 with CPLEX 12 solver. The experiment is run on Intel Core i5, 1.80 GHz PC with 4 GB RAM.

We select two precedence networks with 21 and 30 tasks from Scholl [24] (available in <http://www.assembly-line-balancing.de>).

These well-known test problems are used by Miltenburg and Wijngaard [7], Urban [8] and Aase et al. [9] in their U-shaped studies. For cycle time, the number of the workstations and processing time values, we also benefit from the U-shaped studies of Miltenburg and Wijngaard [7] and Urban [8]. For each N , we use three cycle time (CT) values. For $N=21$, we set the CT values to 14, 15 and 21 and for $N=30$, we set them to 45, 60 and 75. We select K as K_{min} , $K_{min} + 1$ and $K_{min} + 2$ where K_{min} is the minimum number of workstations for a feasible solution. We generate task times from discrete uniform distribution $U[1, 10]$ and $U[1, 21]$ for the 21 and 30 task networks, respectively.

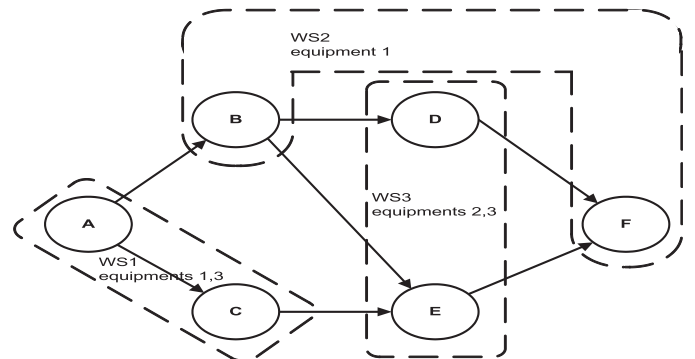


Fig. 5. The optimal task and equipment assignments for the example instance.

Table 2
The BAB algorithm and model performances for $L=3, N=21$.

CT	K	BAB					Model		
		CPU time		Number of nodes		# of unsolved instances	CPU time		# of unsolved instances
		Aver	Max	Aver	Max		Aver	Max	
Set I									
14	8	19.7	47.6	140,945	320,763	0	866.4	1766.2	0
	9	135.5	349.2	979,042	2,566,607	0	3830.6	7200.0	3
	10	1548.5	7200.0	12,444,791	59,826,746	1	4770.0	7200.0	6
15	8	25.0	63.2	199,490	526,380	0	1429.0	3717.0	0
	9	108.0	697.0	891,471	5,760,871	0	3523.8	7200.0	3
	10	189.7	872.5	1,592,683	7,259,671	0	5788.7	7200.0	7
21	6	11.7	53.1	90,033	412,689	0	323.4	770.8	0
	7	20.0	55.4	160,633	456,946	0	1993.2	4740.3	0
	8	22.3	57.6	180,147	458,668	0	5443.3	7200.0	5
Set II									
14	6	21.2	48.6	146,901	320,763	0	802.4	1860.7	0
	7	146.7	383.0	1,025,468	2,721,610	0	3172.5	7200.0	2
	8	1583.1	7200.0	12,545,279	59,544,377	1	4470.7	7200.0	4
15	6	25.6	64.5	200,417	526,380	0	1153.9	2557.0	0
	7	112.2	710.1	908,428	5,771,693	0	4132.6	7200.0	3
	8	199.9	879.6	1,656,551	7,279,621	0	5659.3	7200.0	7
21	6	12.9	51.4	100,926	412,690	0	451.3	1165.9	0
	7	19.7	55.3	160,987	456,947	0	2556.6	5874.4	0
	8	21.6	55.8	179,462	465,223	0	5470.3	7200.0	4

We set the number of equipments, L , to 2, 3 and 5 and generate the following sets to define our equipment costs.

$L=2$ Set I $C_1=300$ $C_2=400$

Set II $C_1=300$ $C_2=400$

$L=3$ Set I $C_1=200$ $C_2=300$ $C_3=400$

Set II $C_1=300$ $C_2=350$ $C_3=400$

$L=5$ Set I $C_1=200$ $C_2=250$ $C_3=300$ $C_4=350$ $C_5=400$

Note that, Set I resides small equipment costs with higher variability and Set II resides large equipment costs with lower variability. We select a single set for $L=5$ as our experimentation on $L=2$ and 3 has revealed that the costs do not influence the performance of the branch and bound algorithm. For each task i , we generate l_i from $U[1, L]$.

Two values of N , three values of CT , three values of K and five sets of L form 90 ($2 \times 3 \times 3 \times 5$) combinations. For each combination, we generate 10 instances. Hence, our experiment set resides 900 problem instances.

We report on the performances of the mathematical model and the branch and bound algorithm in Tables 1–5. Based on different CT and K values and L sets, Tables 1–3 report on $N=21$ instances whereas Tables 4 and 5 report on the instances with $N=30$.

We set a termination limit of 2 h for the execution of the CPLEX model and branch and bound algorithm. We include the instances that do not return optimal solutions in 2 h, while computing the performance statistics.

Our first observation is that the problem size parameters N and K are very effective on the complexity of the solutions. The effect of N is more dominant than that of K . For $N=21$, we could solve instances with up to 10 workstations when $CT=14$ and 15, and 8 workstations when $CT=21$. On the other hand when $N=30$, we could not solve any instance when $K=10$. The maximum numbers of workstations that we find solutions are 10 and 9 for small and big N , respectively.

When N increases, the number of decisions hence the depth of the branch and bound tree increases significantly. This increase directly affects the number of nodes and CPU times. Moreover, as the number of tasks increases, the precedence relations and lower bounds become weaker. The tables indicate that when $N=21, K=8$ and $L=2$ all 60 instances over two sets could be solved in 2 h. When

Table 3
The BAB algorithm and model performances for $L=5, N=21$.

CT	K	BAB					Model		
		CPU time		Number of nodes		# of unsolved instances	CPU time		# of unsolved instances
		Aver	Max	Aver	Max		Aver	Max	
Set I									
14	8	356.7	895.1	1,998,431	5,178,657	0	1560.4	3218.3	0
	9	980.7	2785.3	5,564,119	15,381,393	0	2712.0	7200.0	1
	10	3110.9	7200.0	17,503,778	37,156,171	1	5218.6	7200.0	5
15	8	1145.6	7200.0	5,768,519	35,732,519	1	2591.2	7200.0	1
	9	968.3	4419.3	5,334,528	24,240,089	0	4866.5	7200.0	4
	10	2404.6	7200.0	13,957,032	40,881,837	1	6654.4	7200.0	9
21	6	30.9	108.5	173,234	629,043	0	600.5	933.7	0
	7	145.2	456.7	841,658	2,658,035	0	3970.4	7200.0	1
	8	229.8	553.6	1,367,819	3,359,613	0	6461.8	7200.0	8

Table 4The BAB algorithm and model performances for $L = 2$, $N = 30$.

CT	K	BAB					Model		
		CPU time		Number of nodes		# of unsolved instances	CPU time		# of unsolved instances
		Aver	Max	Aver	Max		Aver	Max	
Set I									
45	8	2907.1	7200.0	21,595,138	53,295,505	1	2589.3	5052.5	0
	9	3991.7	7200.0	29,577,082	54,365,522	3	5950.1	7200.0	5
	10	4241.1	7200.0	31,658,330	56,121,938	4	7171.7	7200.0	9
60	8	137.2	818.5	1,028,169	6,092,860	0	280.4	801.6	0
	9	902.2	7200.0	6,968,110	56,074,976	1	1071.2	2932.1	0
	10	903.4	7200.0	6,902,235	55,226,296	1	3266.5	7200.0	2
75	6	399.7	2317.6	2,957,320	17,045,067	0	115.3	371.0	0
	7	425.2	2409.5	3,177,618	17,936,891	0	169.7	424.4	0
	8	425.2	2411.4	3,178,719	17,946,358	0	441.8	1145.9	0
Set II									
45	6	2945.8	7200.0	21,362,305	52,131,340	1	1675.1	2257.2	0
	7	4006.2	7200.0	29,504,271	54,317,292	3	5589.4	7200.0	4
	8	4235.5	7200.0	31,442,326	55,035,328	4	7200.0	7200.0	10
60	6	138.3	821.3	1,028,638	6,092,860	0	440.1	2038.0	0
	7	903.0	7200.0	6,952,161	55,903,493	1	1100.3	2265.3	0
	8	904.9	7200.0	6,986,385	56,055,774	1	3454.8	7200.0	1
75	6	394.7	2284.7	2,957,320	17,045,067	0	106.3	567.4	0
	7	420.7	2375.6	3,177,618	17,936,891	0	278.9	1278.5	0
	8	422.6	2387.8	3,178,719	17,946,358	0	656.6	2551.1	0

N becomes 30, 6 out of 40 instances with $CT = 45$ and 60 values could not be solved.

For fixed N , when K increases the number of nodes and CPU time increase significantly. This is due to the fact that the feasible region enlarges, thereby leaving many partial solutions for evaluation. Moreover, the evaluations by reduction and bounding mechanisms are more effective for small K .

The results reveal that when the number of equipments increases, the problem becomes harder to solve. This is due to the fact that reduction and bounding mechanisms perform poorer for higher values of L . We also observe that the performance of the algorithm improves with an increase in CT . As CT increases, the total cost

values become relatively small. This follows the associated lower bounds are closer to the optimal cost values, hence they are more powerful.

Note from the tables that in all problem combinations, the BAB shows superior performance than the CPLEX model. The only exception is the combination with $L = 3$, $N = 30$ and $CT = 75$. Note that for $N = 21$, the number of unsolved instances in 2 h (out of 360) is 3 and 64 for the BAB and model, respectively. For example, when $N = 21$, $L = 2$, $CT = 15$, $K = 10$ the average CPU time of the BAB is 24 s whereas the CPLEX model cannot solve 9 instances in 2 h. For $N = 30$, when $CT = 45$, $K = 9$, the CPU times of BAB are 4241.1, 4235.5, 4050.8 and 4440.6 s while the CPU times of the CPLEX model are 7171.7,

Table 5The BAB algorithm and model performances for $L = 3$, $N = 30$.

CT	K	BAB					Model		
		CPU time		Number of nodes		# of unsolved instances	CPU time		# of unsolved instances
		Aver	Max	Aver	Max		Aver	Max	
Set I									
45	7	3609.9	7200.0	20,018,097	39,677,436	1	6434.8	7200.0	3
	8	3960.7	7200.0	22,236,329	42,146,727	2	7200.0	7200.0	10
	9	4050.8	7200.0	23,362,392	42,505,686	2	7200.0	7200.0	10
60	6	2033.3	7200.0	11,354,383	40,351,081	1	1311.8	4385.3	0
	7	3617.2	7200.0	20,377,296	42,371,958	3	4226.3	7200.0	4
	8	3835.6	7200.0	21,930,592	42,614,738	4	6655.0	7200.0	7
75	5	1332.0	3625.8	7,555,611	20,603,608	0	113.5	174.2	0
	6	3043.7	7200.0	17,623,516	41,851,255	2	628.3	1212.5	0
	7	3124.4	7200.0	18,244,150	42,342,744	2	2234.3	7200.0	1
Set II									
45	7	3846.3	7200.0	21,456,888	40,610,375	1	6035.0	7200.0	2
	8	4051.8	7200.0	22,605,721	40,925,976	2	7200.0	7200.0	10
	9	4440.6	7200.0	25,639,153	42,753,978	4	7200.0	7200.0	10
60	6	2023.4	7200.0	11,340,572	40,191,025	1	1738.1	7200.0	1
	7	3598.0	7200.0	20,370,871	41,335,961	3	4687.5	7200.0	4
	8	3823.5	7200.0	21,823,238	41,756,306	4	7091.9	7200.0	9
75	5	1428.1	3699.4	8,067,846	21,049,568	0	180.4	413.3	0
	6	3155.4	7200.0	18,208,051	41,654,819	2	544.0	1383.7	0
	7	3241.5	7200.0	19,097,855	42,738,748	2	1695.2	2845.9	0

7200, 7200 and 7200 s for $L=2$ and Set I, $L=2$ and Set II, $L=3$ and Set I, and $L=3$ and Set II, respectively. Notice that, the CPLEX model could solve only 1 instance optimally (out of 40) in 2 h.

We observe the significant effects of the N , L and K values on the complexity of the CPLEX model. The performance of the model deteriorates significantly as N or K or L increases, as those increases trigger exponential increases in the number of the binary decision variables. The results have also revealed that the cycle time CT has a remarkable effect on the complexity of the CPLEX model. As CT increases, many tasks fit to the same workstation, hence the assignment decisions are taken easier.

6. Conclusions

In this study, we consider a U-shaped assembly line balancing problem with equipment requirements. We assume that the tasks require specified sets of equipments and the cycle time is known. Our problem is to assign the tasks together with their equipments to the workstations so as to minimize the total equipment cost. Making equipment decisions together with the task decisions would assign the tasks that can be performed by the same equipment to the same workstation, hence save from the equipment usages. The problem is theoretically more challenging than its straight line counterpart, due to having more choices for task and equipment assignments to any workstation. To the best of our knowledge, there is no reported optimization algorithm to solve the equipment assignment problem in U-shaped lines.

We formulate the problem as a mixed integer linear model and observe that the model is limited to solve small-sized problem instances and the solution times increase considerably with increases in the number of the equipments.

We develop some properties that explain the structure of the optimal solution. Based on these properties, we define an efficient way of enumerating the partial solutions of our branch and bound algorithm. We improve the performance of the branch and bound algorithm via some efficient lower bounding procedures.

The results of our extensive experiment have also revealed that the number of tasks, the number of workstations and cycle time values are very effective on the performance of the branch and bound algorithm. However, we find that an increase in the number of equipments slightly increases the solution times. The algorithm can solve the problem instances up to 21 tasks and 10 workstations, and 30 tasks and 8 workstations, in 2 h. Miltenburg and Wijngaard [7] state that most U-shaped assembly lines consist of less than 30 tasks and 10 stations. We hope that our algorithm is used by the practitioners that operate U-shaped lines with several equipment types.

The future research may point out the development of the heuristic procedures to solve large-sized problem instances. The heuristic procedures like truncated branch and bound, α -approximation algorithms and beam search techniques, may benefit from the ideas of our branch and bound algorithm. The future research may also consider the development of new branch

and bound algorithms for more complex U-shaped lines. For example, the branch, bound and remember type algorithms that have been recently proposed for simple assembly line balancing problems (see [21,22]) might provide a promising perspective.

We may extend our work to the resource dependent assembly line balancing problem that additionally resides equipment selection decisions.

Acknowledgements

The authors are grateful to the anonymous referees for their constructive and helpful comments.

References

- [1] Battaia O, Dolgui A. A taxonomy of line balancing problems and their solution approaches. *Int J Prod Econ* 2013;142:259–77.
- [2] Miltenburg J. U-shaped production lines: a review of theory and practice. *Int J Prod Econ* 2001;70:201–14.
- [3] Monden Y. Toyota production system. Norcross: Institute of Industrial Engineers; 1993.
- [4] Ağpak K, Gökçen H. Assembly line balancing: two resource constrained cases. *Int J Prod Econ* 2005;96:129–40.
- [5] Kara Y, Özgüven C, Yalcin N, Atasagun Y. Balancing straight and U-shaped assembly lines with resource dependent task times. *Int J Prod Res* 2012;49:6387–405.
- [6] Faaland BH, Klastorin TD, Schmitt TG, Shtub A. Assembly line balancing with resource dependent setup times. *Decis Sci* 1992;23:343–64.
- [7] Miltenburg J, Wijngaard J. The U-line line balancing problem. *Manage Sci* 1994;40:1378–88.
- [8] Urban TL. Optimal balancing of U-shaped lines. *Manage Sci* 1998;44:738–41.
- [9] Aase GR, Schniederjans MJ, Olson JR. U-Opt: an analysis of exact U-shaped line balancing procedures. *Int J Prod Res* 2003;41:4185–210.
- [10] Erel E, Sabuncuoğlu I, Aksu BA. Balancing of U-type assembly systems using simulated annealing. *Int J Prod Res* 2001;39:3003–15.
- [11] Gökçen H, Ağpak K. A goal programming approach to simple U-line balancing problem. *Eur J Oper Res* 2006;171:577–85.
- [12] Lian K, Zhang C, Gao L, Shao X. A modified colonial competitive algorithm for the mixed model U-line balancing and sequencing problem. *Int J Prod Res* 2012;50:5117–31.
- [13] Hamzadayi A, Yildiz G. A genetic algorithm based approach for simultaneously balancing and sequencing of mixed model U-lines with parallel workstations and zoning constraints. *Comput Ind Eng* 2012;62:206–15.
- [14] Buckhin J, Tzur M. Design of flexible assembly line to minimize equipment cost. *IIE Trans* 2000;32:585–98.
- [15] Buckhin J, Rubinovitz J. A weighted approach for assembly line design with station paralleling and equipment selection. *IIE Trans* 2003;35:73–85.
- [16] Barutcuoğlu Ş, Azizoglu M. Flexible assembly line design problem with fixed number of workstations. *Int J Prod Res* 2011;49:3691–714.
- [17] Ege Y, Azizoglu M, Özdemirel NE. Assembly line balancing with station paralleling. *Comput Ind Eng* 2009;57:1218–25.
- [18] Yoosefelahe A, Aminnayeri M, Mosadegh H, Davari Ardakani H. Type II robotic assembly line balancing problem: an evolution strategies algorithm for a multi-objective model. *J Manuf Syst* 2012;31:139–51.
- [19] Jayaswal S, Agarwal P. Balancing U-shaped assembly lines with resource dependent task times: a simulated annealing approach. *J Manuf* 2014;33:522–34.
- [20] Baybars İ. A survey of exact algorithms for the simple assembly line balancing problem. *Manage Sci* 1986;32:909–32.
- [21] Sewell EC, Jacobson SH. A branch, bound, and remember algorithm for the simple assembly line balancing problem. *INFORM J Comput* 2012;24:433–42.
- [22] Morrison DR, Sewell EC, Jacobson SH. An application of the branch, bound, and remember algorithm to a new simple assembly line balancing dataset. *Eur J Oper Res* 2014;236:403–9.
- [24] Scholl A. Data for U-shaped assembly line balancing problems; 2007. <http://www.assembly-line-balancing.de/>